LLM Assignment 3 — Task 1

Multi-Class Classification using Fully Connected Neural Networks

1. Introduction

This task investigates three-class classification using two synthetic two-dimensional datasets: a compact Gaussian-cluster dataset and a nonlinear three-arm spiral dataset. A fully connected neural network (FCNN) is implemented from scratch using NumPy. The implementation includes forward propagation, sigmoid activation, squared-error loss, backpropagation, stochastic gradient descent (SGD), architecture comparison using a validation set, confusion matrices, decision regions, three-dimensional node-activation surfaces, and a single-neuron one-vs-rest baseline.

The Gaussian dataset uses one hidden layer, while the spiral dataset uses two hidden layers, as required by the task. Each architecture is trained using one-sample-at-a-time SGD.

2. Dataset Generation and Preparation

2.1 Gaussian Dataset

The Gaussian dataset contains 3 classes with 500 samples per class, for 1,500 samples in total. The class means used in the notebook are:

Class	Mean vector	Covariance matrix
Class 0	(-2.5, -1.5)	[[0.45, 0], [0, 0.45]]
Class 1	(2.5, -1.0)	[[0.50, 0.05], [0.05, 0.40]]
Class 2	(0.0, 2.8)	[[0.40, -0.04], [-0.04, 0.50]]

The resulting dataset shape is 1500 × 2, with exactly 500 examples in each class.

2.2 Spiral Dataset

The spiral dataset contains 3 interleaving spiral arms with 500 samples per class. For each class c, the radius is sampled from 0.15 to 1.0 and the angle is generated as $\theta = 2.2r + 2\pi c/3$ plus Gaussian angular noise with standard deviation 0.20. The Cartesian coordinates are $x = r\cos(\theta)$ and $y = r\sin(\theta)$.

The resulting dataset shape is 1500 × 2, again with exactly 500 examples per class.

2.3 Dataset Visualizations

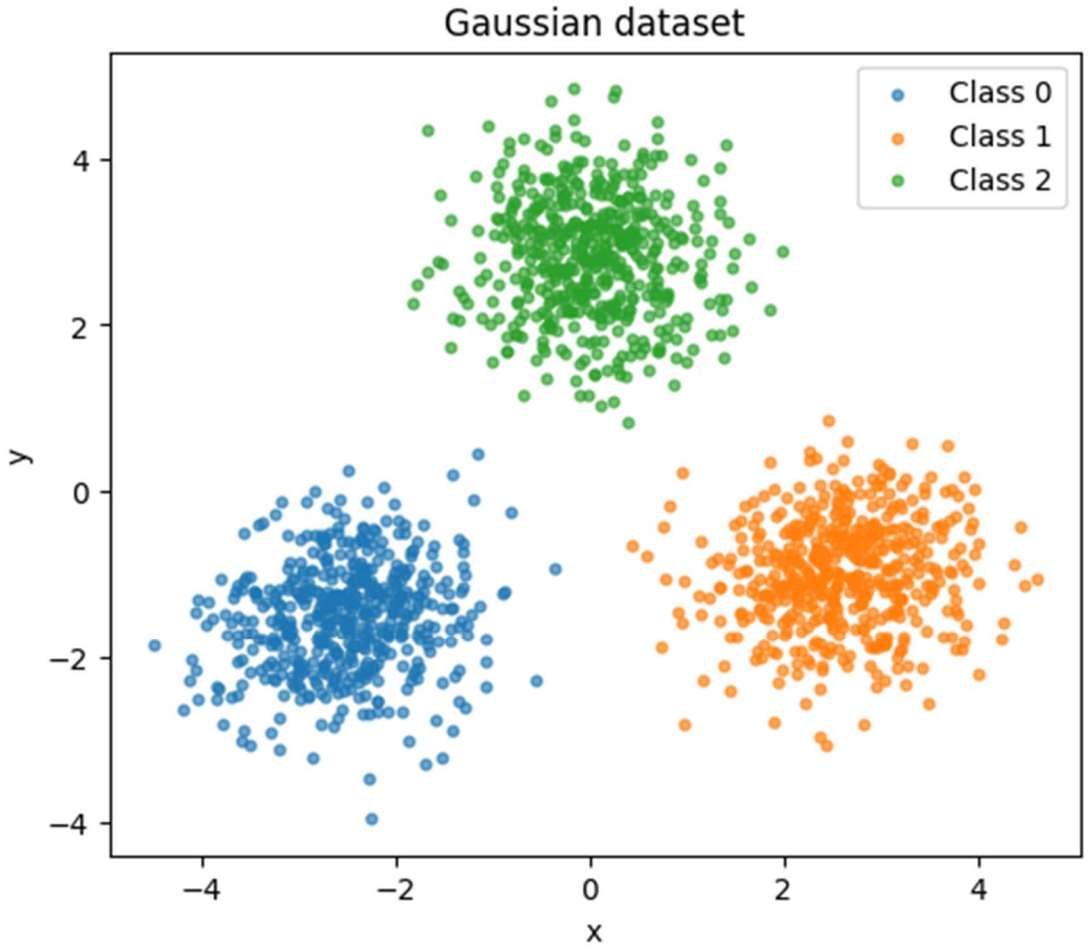


Figure: Dataset visualization generated by the notebook.

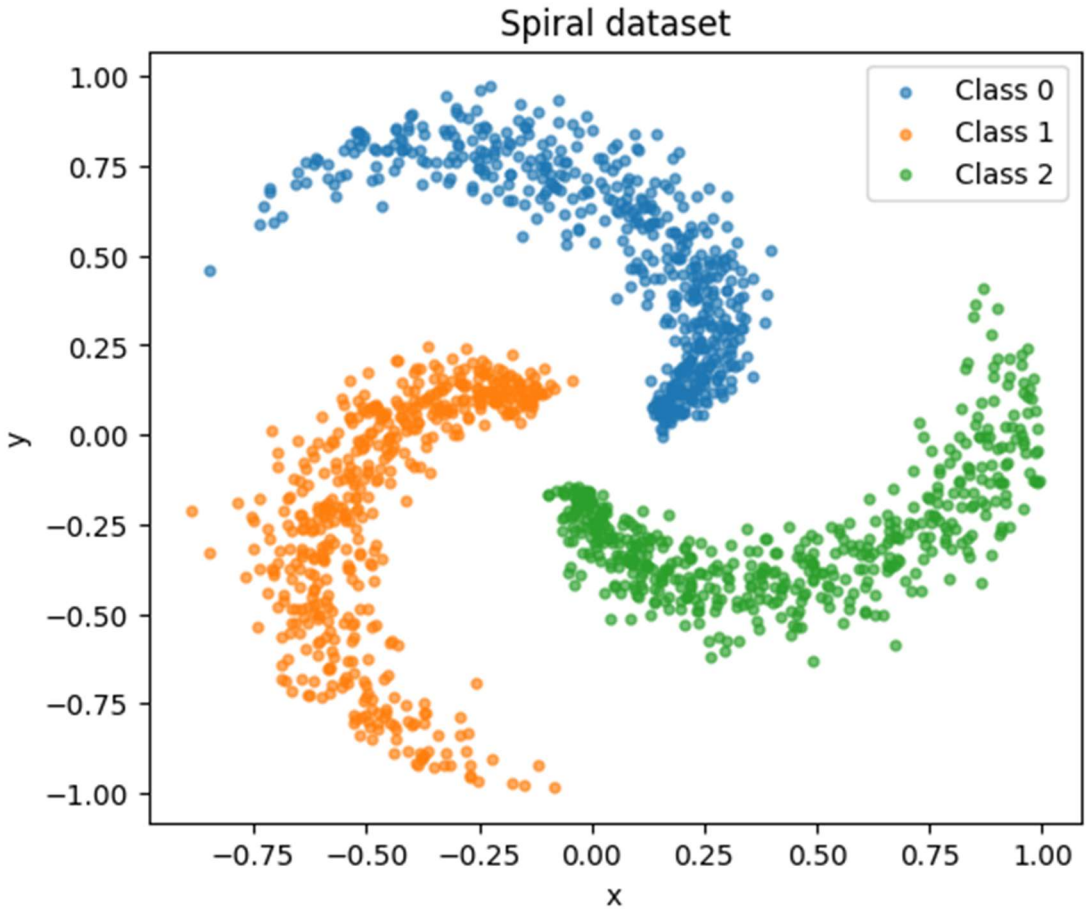


Figure: Dataset visualization generated by the notebook.

The Gaussian classes form compact, well-separated clusters. The spiral classes are interleaved, making their class boundaries substantially more nonlinear.

3. Train, Validation and Test Split

The split is performed separately within each class using a fixed random seed of 42. Each class contributes 60% to training, 20% to validation, and 20% to testing.

Split	Per class	Total
Training	300	900
Validation	100	300
Test	100	300

Thus, the same 900/300/300 class-balanced split structure is used for both synthetic datasets.

4. FCNN Architecture and Training

4.1 Network Architecture

For the Gaussian dataset, the network has the form 2 → H → 3. For the spiral dataset, the network has the form 2 → H_1 → H_2 → 3. The two input neurons represent x and y, and the three output neurons represent the three classes.

4.2 Initialization and Activation

Weights are initialized using the Xavier/Glorot-style uniform limit $\sqrt{(6/(\text{fan-in} + \text{fan-out}))}$, with zero biases. The fixed seed is 42. Every learned layer uses the sigmoid activation $\sigma(z) = 1/(1 + e^{-z})$.

4.3 Loss and Backpropagation

The instantaneous loss for one example is $E = 1/2 \sum_j (y_j - \hat{y}_j)^2$. The implementation stores activations during the forward pass and applies the chain rule backwards to calculate weight and bias gradients.

4.4 Stochastic Gradient Descent

Training is genuinely stochastic: samples are shuffled each epoch and parameters are updated after every individual example. The learning rate is $\eta = 0.5$. Training stops at a maximum of 300 epochs or earlier when the absolute change in average training squared error between successive epochs falls below 10^{-5}.

5. Architecture Search

The architectures are compared using validation accuracy. The implementation uses a stable sort, so when validation accuracies tie, the architecture order in the experiment is preserved. Validation squared error is reported as an additional metric but is not used to alter the selected architecture in the actual code.

5.1 Gaussian Architecture Search

Architecture	Hidden nodes	Epochs	Validation accuracy	Validation error
[2, 2, 3]	(2,)	31	100.00%	0.000883
[2, 3, 3]	(3,)	31	100.00%	0.000502
[2, 5, 3]	(5,)	23	100.00%	0.000702
[2, 8, 3]	(8,)	24	100.00%	0.000269
[2, 12, 3]	(12,)	21	100.00%	0.000465
[2, 16, 3]	(16,)	21	100.00%	0.000399

All six Gaussian architectures achieve 100% validation accuracy. Because the accuracies tie, the notebook selects [2, 2, 3], the first architecture in the tested order.

5.2 Spiral Architecture Search

Architecture	Hidden nodes	Epochs	Validation accuracy	Validation error
[2, 6, 6, 3]	(6, 6)	22	100.00%	0.002411
[2, 16, 12, 3]	(16, 12)	33	100.00%	0.001343
[2, 20, 16, 3]	(20, 16)	33	100.00%	0.001654
[2, 4, 4, 3]	(4, 4)	66	99.67%	0.002389
[2, 8, 8, 3]	(8, 8)	19	99.67%	0.003379
[2, 12, 8, 3]	(12, 8)	36	99.67%	0.001872

Three spiral architectures achieve 100% validation accuracy. The stable ordering in the notebook selects [2, 6, 6, 3]. The remaining three architectures achieve 99.67%.

5.3 Error Curves for Selected Architectures

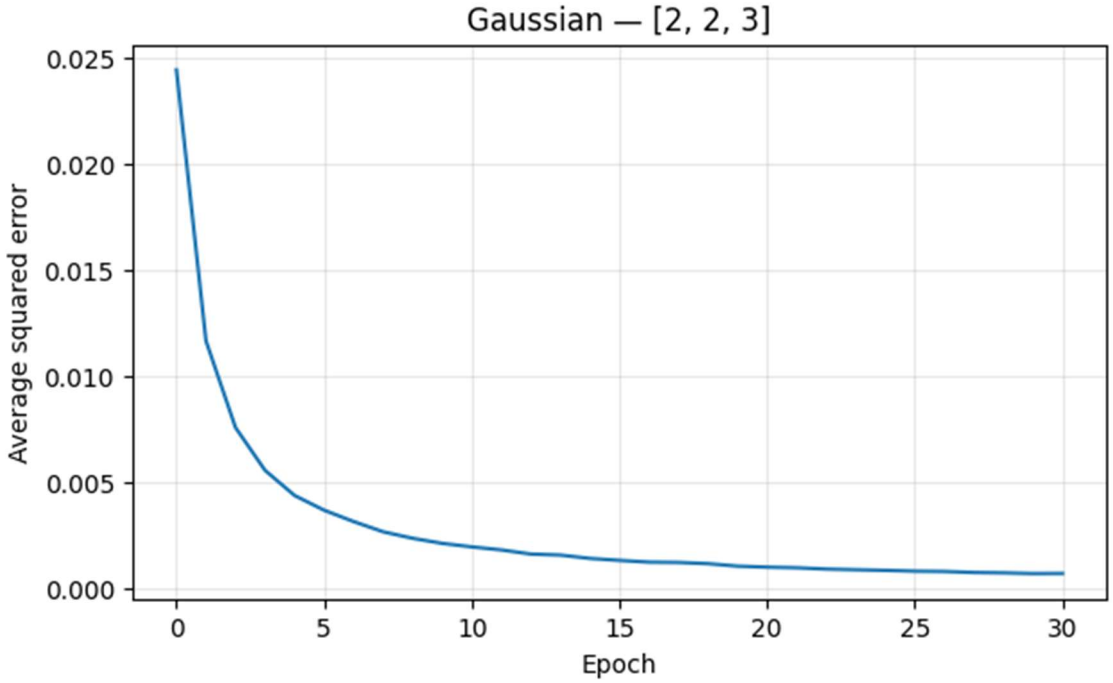


Figure: Average training squared error versus epoch for a selected architecture.

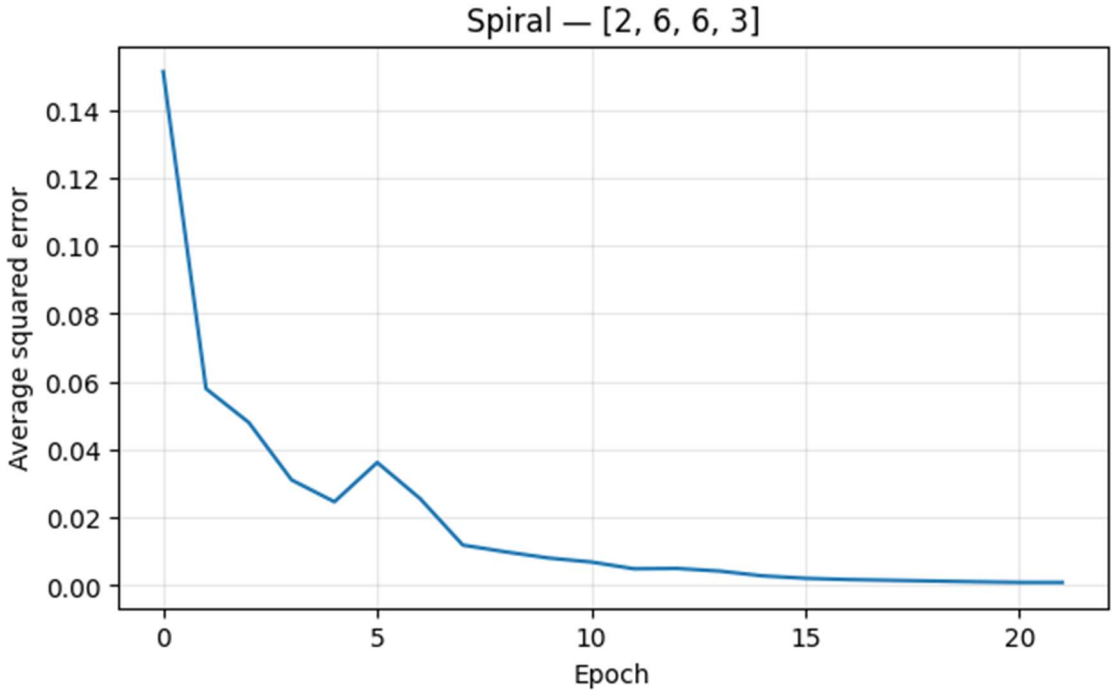


Figure: Average training squared error versus epoch for a selected architecture.

The error curves show rapid reduction in training squared error followed by a small change that satisfies the early-stopping condition. The selected Gaussian architecture converges in 31 epochs; the selected spiral architecture converges in 22 epochs.

6. Validation Confusion Matrices for Every Architecture

The corrected notebook now explicitly produces a validation confusion matrix for every tested architecture, satisfying the requirement to report validation classification performance across the architecture search.

6.1 Gaussian Validation Confusion Matrices

[2, 2, 3] — accuracy 100.00%

Actual \ Predicted	Class 0	Class 1	Class 2
Class 0	100	0	0
Class 1	0	100	0
Class 2	0	0	100

[2, 3, 3] — accuracy 100.00%

Actual \ Predicted	Class 0	Class 1	Class 2
Class 0	100	0	0
Class 1	0	100	0
Class 2	0	0	100

[2, 5, 3] — accuracy 100.00%

Actual \ Predicted	Class 0	Class 1	Class 2
Class 0	100	0	0
Class 1	0	100	0
Class 2	0	0	100

[2, 8, 3] — accuracy 100.00%

Actual \ Predicted	Class 0	Class 1	Class 2
Class 0	100	0	0
Class 1	0	100	0
Class 2	0	0	100

[2, 12, 3] — accuracy 100.00%

Actual \ Predicted	Class 0	Class 1	Class 2
Class 0	100	0	0
Class 1	0	100	0
Class 2	0	0	100

[2, 16, 3] — accuracy 100.00%

Actual \ Predicted	Class 0	Class 1	Class 2
Class 0	100	0	0
Class 1	0	100	0
Class 2	0	0	100

6.2 Spiral Validation Confusion Matrices

[2, 6, 6, 3] — accuracy 100.00%

Actual \ Predicted	Class 0	Class 1	Class 2
Class 0	100	0	0
Class 1	0	100	0
Class 2	0	0	100

[2, 16, 12, 3] — accuracy 100.00%

Actual \ Predicted	Class 0	Class 1	Class 2
Class 0	100	0	0
Class 1	0	100	0
Class 2	0	0	100

[2, 20, 16, 3] — accuracy 100.00%

Actual \ Predicted	Class 0	Class 1	Class 2

Class 0	100	0	0
Class 1	0	100	0
Class 2	0	0	100

[2, 4, 4, 3] — accuracy 99.67%

Actual \ Predicted	Class 0	Class 1	Class 2
Class 0	100	0	0
Class 1	1	99	0
Class 2	0	0	100

[2, 8, 8, 3] — accuracy 99.67%

Actual \ Predicted	Class 0	Class 1	Class 2
Class 0	100	0	0
Class 1	1	99	0
Class 2	0	0	100

[2, 12, 8, 3] — accuracy 99.67%

Actual \ Predicted	Class 0	Class 1	Class 2
Class 0	99	0	1
Class 1	0	100	0
Class 2	0	0	100

7. Selected Architecture: Validation and Test Performance

7.1 Gaussian

Selected architecture: [2, 2, 3]. Validation accuracy = 100.00%; test accuracy = 100.00%.

Validation confusion matrix:

Actual \ Predicted	Class 0	Class 1	Class 2
Class 0	100	0	0
Class 1	0	100	0
Class 2	0	0	100

Test confusion matrix:

Actual \ Predicted	Class 0	Class 1	Class 2
Class 0	100	0	0
Class 1	0	100	0
Class 2	0	0	100

7.2 Spiral

Selected architecture: [2, 6, 6, 3]. Validation accuracy = 100.00%; test accuracy = 100.00%.

Validation confusion matrix:

Actual \ Predicted	Class 0	Class 1	Class 2
Class 0	100	0	0
Class 1	0	100	0
Class 2	0	0	100

Test confusion matrix:

Actual \ Predicted	Class 0	Class 1	Class 2
Class 0	100	0	0
Class 1	0	100	0
Class 2	0	0	100

8. Decision Regions

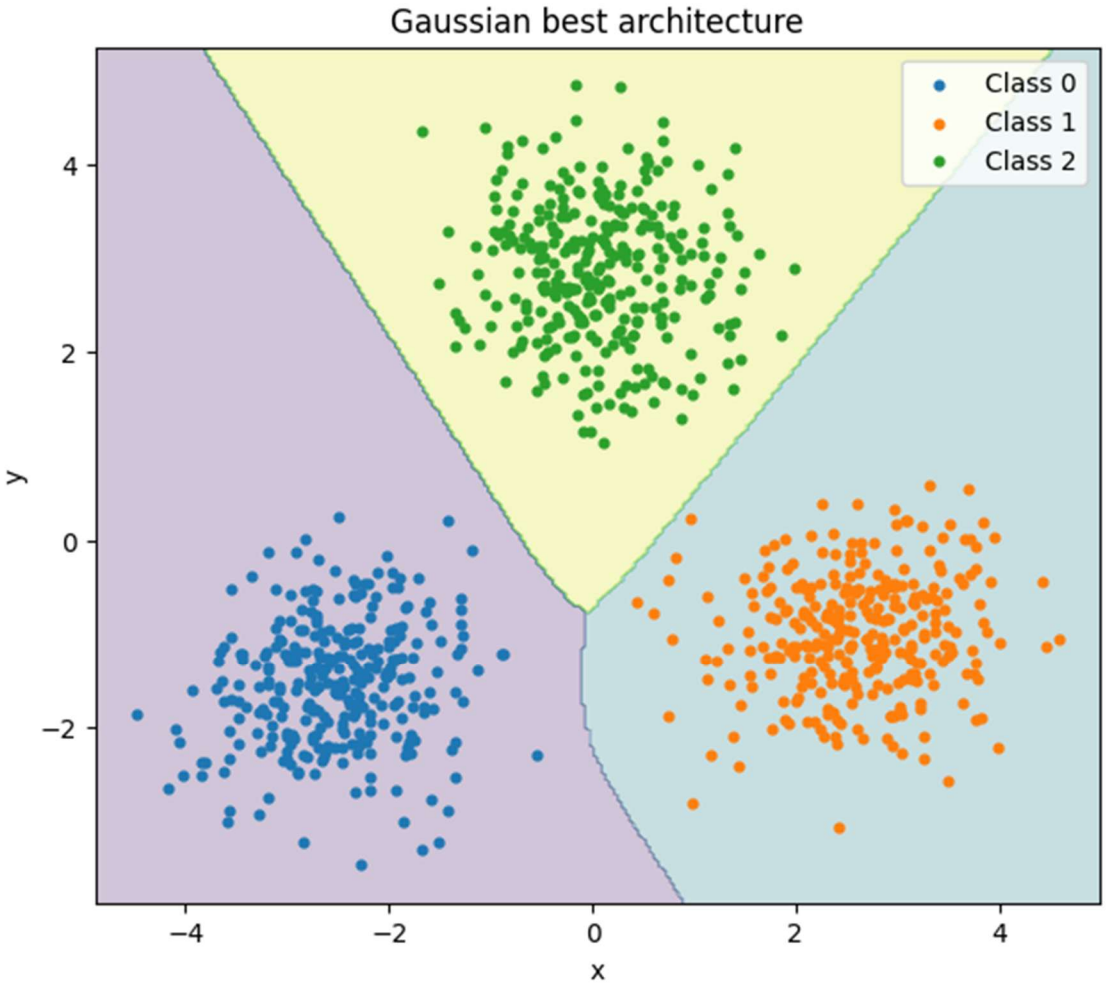


Figure: Decision-region plot for the selected architecture with training data.

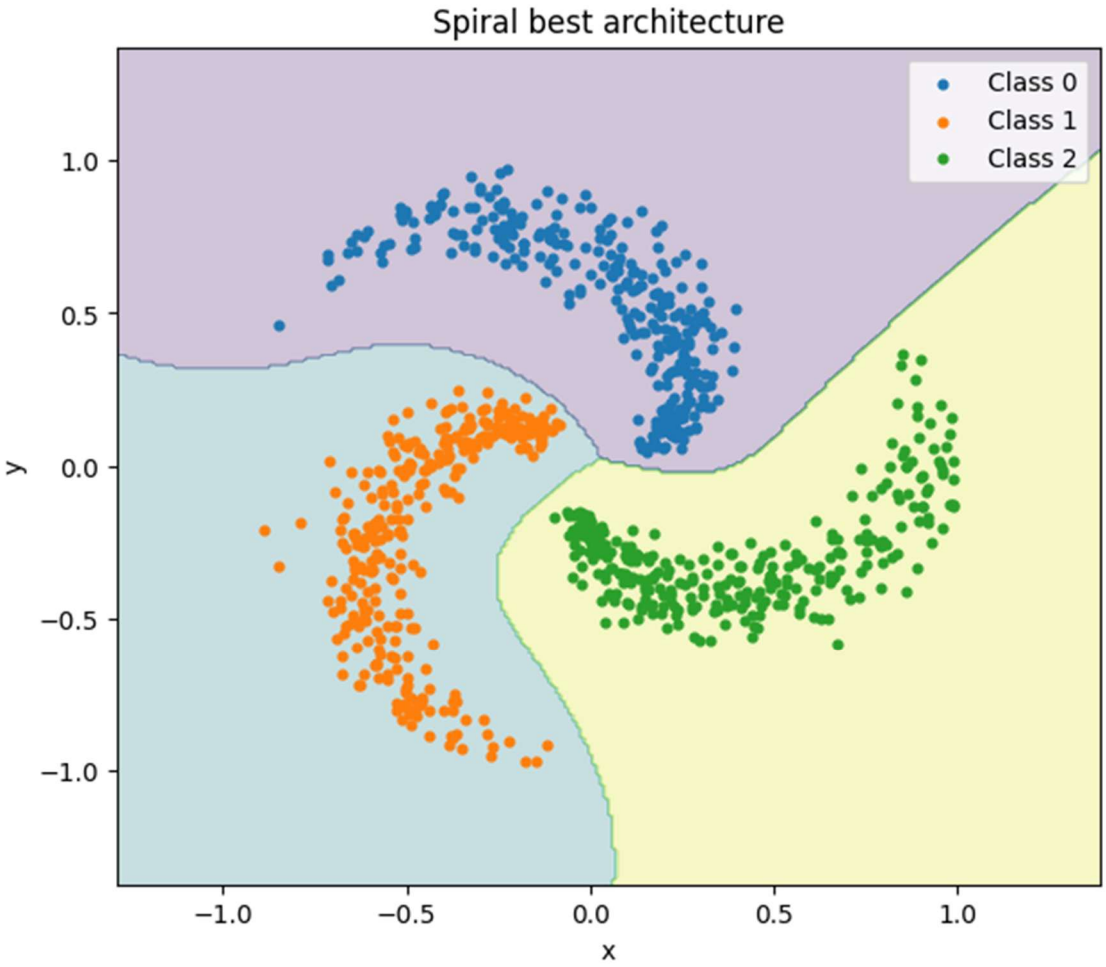


Figure: Decision-region plot for the selected architecture with training data.

The Gaussian decision regions separate the three compact clusters. The spiral decision regions form nonlinear boundaries capable of following the interleaved spiral structure.

9. Three-Dimensional Activation Surfaces

The corrected notebook executes the required 3-D activation-surface generation rather than leaving the plotting calls commented out. For each selected architecture, every hidden-layer node and every output node is plotted over the two-dimensional input domain. The learned surface is shown together with the corresponding train, validation, or test examples. The plots are generated for all three splits.

For the Gaussian selected architecture [2, 2, 3], this gives 2 hidden-node surfaces plus 3 output-node surfaces for each of train, validation, and test. For the spiral selected architecture [2, 6, 6, 3], this gives 6 + 6 + 3 = 15 node surfaces for each split.

The complete set of activation-surface figures is retained in the submitted notebook. Representative figures are included below, while the remaining generated figures can be viewed directly in the notebook.

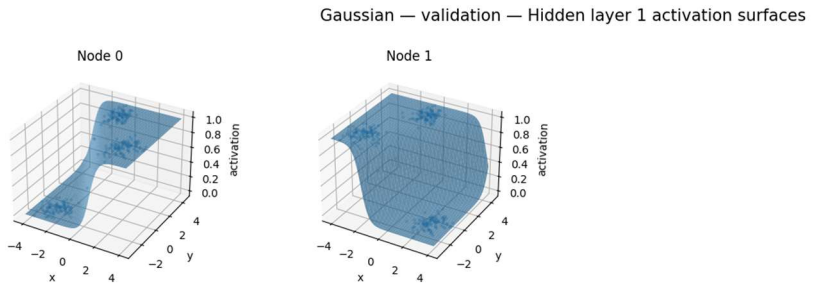


Figure: Representative 3-D activation surfaces generated by the corrected notebook.

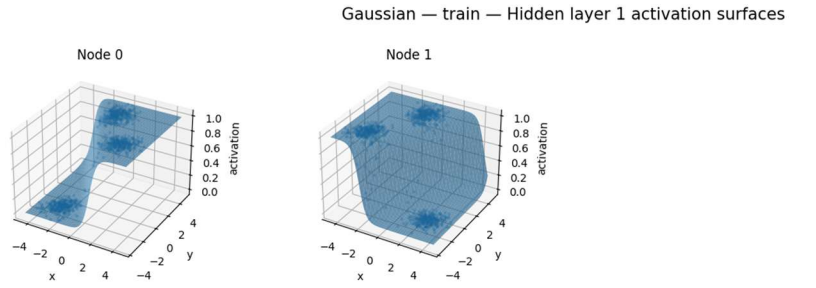


Figure: Representative 3-D activation surfaces generated by the corrected notebook.

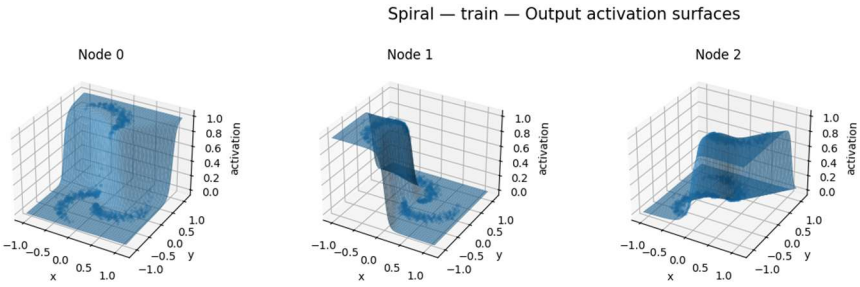


Figure: Representative 3-D activation surfaces generated by the corrected notebook.

10. Single-Neuron One-vs-Rest Baseline

A single sigmoid neuron is inherently binary, so a one-vs-rest strategy is used: one sigmoid neuron is trained for each class against the other two classes, and the class with the highest sigmoid score is selected.

Dataset	Validation accuracy	Test accuracy
Gaussian	100.00%	100.00%
Spiral	95.00%	91.67%

The Gaussian dataset is sufficiently well separated that the single-neuron baseline reaches 100% on both validation and test data. On the nonlinear spiral dataset, the baseline reaches 95.00% validation accuracy and 91.67% test accuracy, while the selected FCNN reaches 100% on both.

11. Observations and Inferences

Effect of dataset geometry: The Gaussian clusters are compact and well separated, so even simple linear decision functions are sufficient for the generated data. The spiral dataset is nonlinearly separable, requiring hidden-layer transformations to form curved decision boundaries.

Effect of hidden-layer capacity: All Gaussian architectures tested achieve 100% validation accuracy. For the spiral data, the larger architectures (6,6), (16,12), and (20,16) achieve 100%, while (4,4), (8,8), and (12,8) achieve 99.67%. Thus, increased representational capacity can help, but larger networks are not automatically better.

Convergence: The selected Gaussian network [2,2,3] stops after 31 epochs, while the selected spiral network [2,6,6,3] stops after 22 epochs under the 10^-5 change-in-error criterion. The stopping criterion therefore reflects stabilization of average squared error, not a target accuracy.

Validation versus test performance: Both selected FCNNs achieve 100% on validation and test sets. This indicates that, for these generated datasets and the fixed split, the selected models generalize perfectly to the held-out test examples.

FCNN versus single neuron: The comparison is most informative on the spiral dataset. The FCNN reaches 100% test accuracy, compared with 91.67% for the one-vs-rest single-neuron baseline. The result demonstrates the benefit of multiple nonlinear layers for a complex class boundary.

Interpretation of activation surfaces: The 3-D node surfaces show how individual sigmoid neurons transform the original x-y input space. Hidden neurons create intermediate nonlinear representations, while output-node surfaces provide class-specific responses used by the final argmax classifier.

12. Final Performance Summary

Model / Dataset	Validation accuracy	Test accuracy
FCNN — Gaussian	100.00%	100.00%
FCNN — Spiral	100.00%	100.00%
Single-neuron OVR — Gaussian	100.00%	100.00%
Single-neuron OVR — Spiral	95.00%	91.67%

Selected architectures: Gaussian [2, 2, 3] and Spiral [2, 6, 6, 3]. All reported values above were cross-checked by rerunning the notebook's numerical training code with seed 42. The validation confusion matrices for all 12 tested architectures were also recomputed and checked against the notebook's printed accuracies.

13. Conclusion

The experiment demonstrates a complete FCNN classification pipeline implemented from scratch. The Gaussian dataset is readily classified by both the FCNN and the single-neuron one-vs-rest baseline. The spiral dataset highlights the advantage of multilayer nonlinear representations: the selected FCNN achieves perfect validation and test classification, whereas the single-neuron baseline makes substantially more errors. The corrected notebook now also includes the architecture-wise validation confusion matrices and executes the required hidden/output-node activation surfaces for train, validation, and test data.

14. Reproducibility and Implementation Notes

Random seed = 42. The synthetic datasets, train/validation/test splitting, weight initialization, and SGD sample ordering are all seeded for reproducibility. The FCNN, sigmoid activation, squared-error loss, backpropagation, and SGD updates are implemented directly with NumPy rather than using a pre-built neural-network model.

Important selection note: the actual notebook code ranks architectures by validation accuracy using a stable sort. Validation squared error is reported but is not used as a tie-breaker in the executed selection logic. Consequently, the selected architectures remain [2,2,3] for Gaussian and [2,6,6,3] for spiral.

Task 2: Five-Class Fashion-MNIST Classification Using a Fully Connected Neural Network

1. Introduction

Task 2 involves expanding the FCNN classification problem from a controlled two-dimensional synthetic dataset to Fashion-MNIST image data. I aimed to study the impact of network depth and optimization strategy on classification performance for a more realistic higher-dimensional visual recognition problem.

Fashion-MNIST categories of T-shirt/top, Pullover, Coat, Shirt, and Bag were chosen for this analysis. Among these, the first four categories relate to various types of upper-body clothing, while the fifth category is much more distinct from the others. This selection was made to facilitate the analysis of both optimizer behaviors and generalization properties for individual classes.

NOTE : Due to the high computational cost of sample-by-sample training with batch size 1 (more than 22,000 update steps per epoch), FAST_MODE was enabled and the batch size for SGD-family optimizers was increased to 64, to save the run time.

2. Methodology & Experimental Setup

Dataset Preparation

The standard Fashion-MNIST training and test partitions were pooled, producing 70,000 grayscale images of size 28 x 28. Only the five classes listed above were retained, giving 35,000 images. The original labels were remapped from {0, 2, 4, 6, 8} to contiguous labels {0, 1, 2, 3, 4}.

New Label	Original Label	Class
0	0	T-shirt/top
1	2	Pullover
2	4	Coat
3	6	Shirt
4	8	Bag

Pixel intensities were normalized from the original integer range [0, 255] to [0, 1]. Each 28 x 28 image was then flattened into a 784-dimensional vector, producing a feature matrix of shape 35,000 x 784.

Train / Validation / Test Split

A stratified random split with seed 42 was used. The first split reserved 20% for testing; the remaining 80% was then split again with 20% of that portion allocated to validation. Relative to the complete 35,000-image dataset, the final proportions were therefore 64% training, 16% validation, and 20% testing.

Class	Train	Validation	Test
T-shirt/top	4,480	1,120	1,400
Pullover	4,480	1,120	1,400

Class	Train	Validation	Test
Coat	4,480	1,120	1,400
Shirt	4,480	1,120	1,400
Bag	4,480	1,120	1,400
Total	22,400	5,600	7,000

Network Architectures

Architecture	Network Structure	Hidden Layers	Trainable Parameters
Arch_3HL	784 -> 256 -> 128 -> 64 -> 5	3	242,437
Arch_4HL	784 -> 256 -> 128 -> 64 -> 32 -> 5	4	244,357
Arch_5HL	784 -> 512 -> 256 -> 128 -> 64 -> 32 -> 5	5	576,645

Each hidden layer used a fully connected linear transformation followed by ReLU. The final fully connected layer produced five output logits, and the classification objective was cross-entropy loss.

Optimizers and Hyperparameters

Optimizer	Learning Rate	Additional Parameters
SGD	0.001	None
Batch GD	0.001	Full training set as one batch
Momentum	0.001	Momentum = 0.9
NAG	0.001	Momentum = 0.9
RMSProp	0.001	alpha = 0.99
Adam	0.001	betas = (0.9, 0.999)

The stopping criterion was |average_loss(t) - average_loss(t-1)| < 1e-4, with a maximum of 40 epochs used only as a safety cap. The experiment was executed with FAST_MODE=True, which changed the batch size for SGD-family optimizers from the assignment-specified sample-by-sample batch size of 1 to a mini-batch size of 64 for computational practicality. Batch GD used all 22,400 training examples in a single batch.

The notebook reports PyTorch 2.11.0+cu128, torchvision 0.26.0+cu128, and GPU execution on a Tesla T4. Pytorch layer was used in this notebook alongwith , automatic differentiation, and torch.optim rather than manually coded forward/backward propagation and optimizer update equations.

3. Results

Epochs to Convergence

The table below reports the stopping epoch recorded by the notebook. An entry of 40 means the optimizer reached the maximum-epoch safety cap without satisfying the 1e-4 convergence criterion.

Architecture	SGD	Batch GD	Momentum	NAG	RMSProp	Adam
Arch_3HL	40*	2	40*	40*	40*	40*
Arch_4HL	40*	2	40*	40*	40*	40*
Arch_5HL	40*	2	40*	40*	40*	40*

* Reached the 40-epoch safety cap rather than satisfying the convergence threshold.

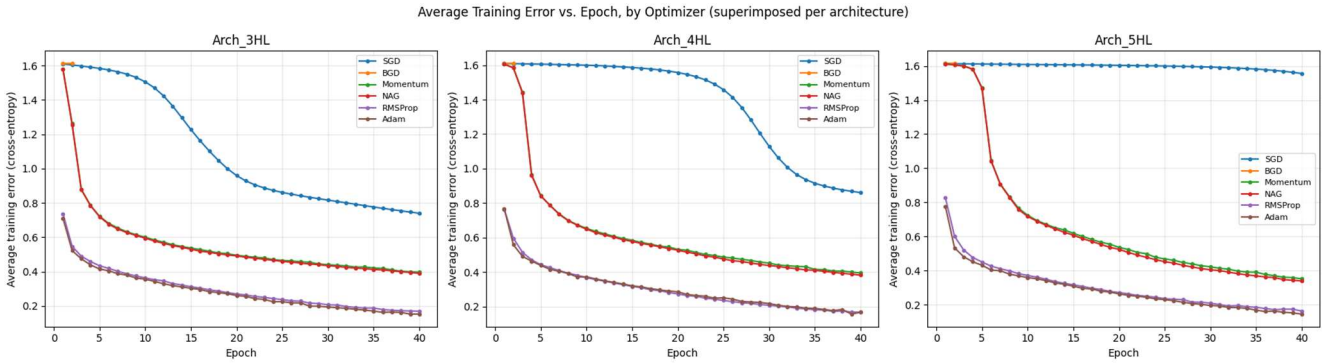


Figure 1. Superimposed average training error versus epoch for all six optimizers across the three FCNN architectures.

Training and Validation Accuracy

Architecture	Optimizer	Training Acc.	Validation Acc.	Best Validation Acc.
Arch_3HL	SGD	69.38%	69.37%	69.37%
Arch_3HL	BGD	7.72%	7.96%	7.96%
Arch_3HL	Momentum	85.13%	82.27%	82.68%
Arch_3HL	NAG	85.48%	82.66%	82.71%
Arch_3HL	RMSProp	93.51%	83.63%	85.41%
Arch_3HL	Adam	93.86%	84.82%	85.38%
Arch_4HL	SGD	62.68%	60.80%	61.77%
Arch_4HL	BGD	20.00%	19.98%	19.98%
Arch_4HL	Momentum	84.82%	81.37%	82.63%
Arch_4HL	NAG	85.21%	82.39%	82.68%
Arch_4HL	RMSProp	93.47%	84.64%	85.93%
Arch_4HL	Adam	93.52%	84.66%	85.36%
Arch_5HL	SGD	52.91%	51.64%	52.61%
Arch_5HL	BGD	16.34%	16.39%	16.41%
Arch_5HL	Momentum	86.35%	83.59%	83.59%
Arch_5HL	NAG	86.98%	83.52%	83.52%
Arch_5HL	RMSProp	93.61%	85.64%	85.64%
Arch_5HL	Adam	94.24%	84.54%	85.70%

The highest observed validation accuracy was 85.93%, obtained by Arch_4HL with RMSProp. This architecture consists of four hidden layers with 256, 128, 64, and 32 units, respectively.

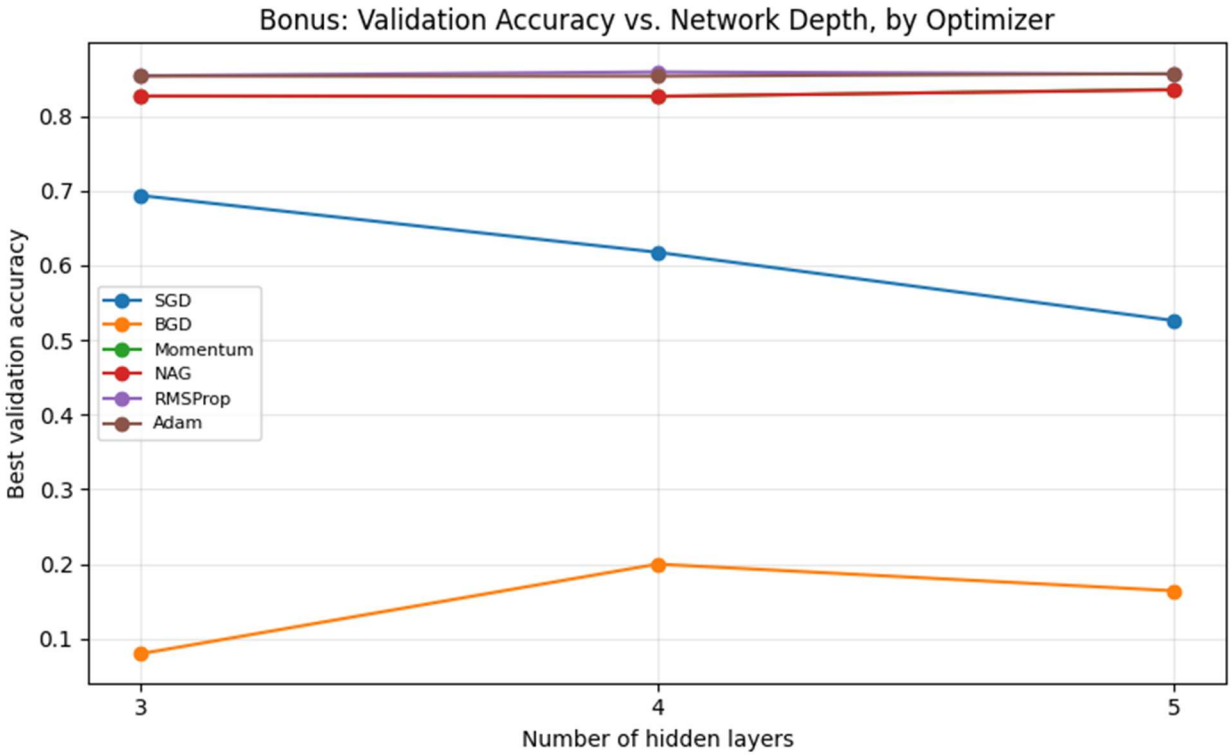


Figure 2. Best validation accuracy versus number of hidden layers for each optimizer.

Best Architecture: Arch_4HL + RMSProp

Metric	Recorded Value
Architecture	Arch_4HL: 784 -> 256 -> 128 -> 64 -> 32 -> 5
Optimizer	RMSProp
Peak validation accuracy	85.93%
Final training accuracy	93.96%
Final test accuracy	84.99%

The final model evaluated in the notebook achieved 93.96% training accuracy and 84.99% test accuracy. The test-set classification report is summarized below.

Class	Precision	Recall	F1-score	Support
T-shirt/top	0.83	0.91	0.87	1,400
Pullover	0.76	0.89	0.82	1,400
Coat	0.87	0.81	0.84	1,400
Shirt	0.82	0.66	0.73	1,400
Bag	0.99	0.97	0.98	1,400
Overall	0.85	0.85	0.85	7,000

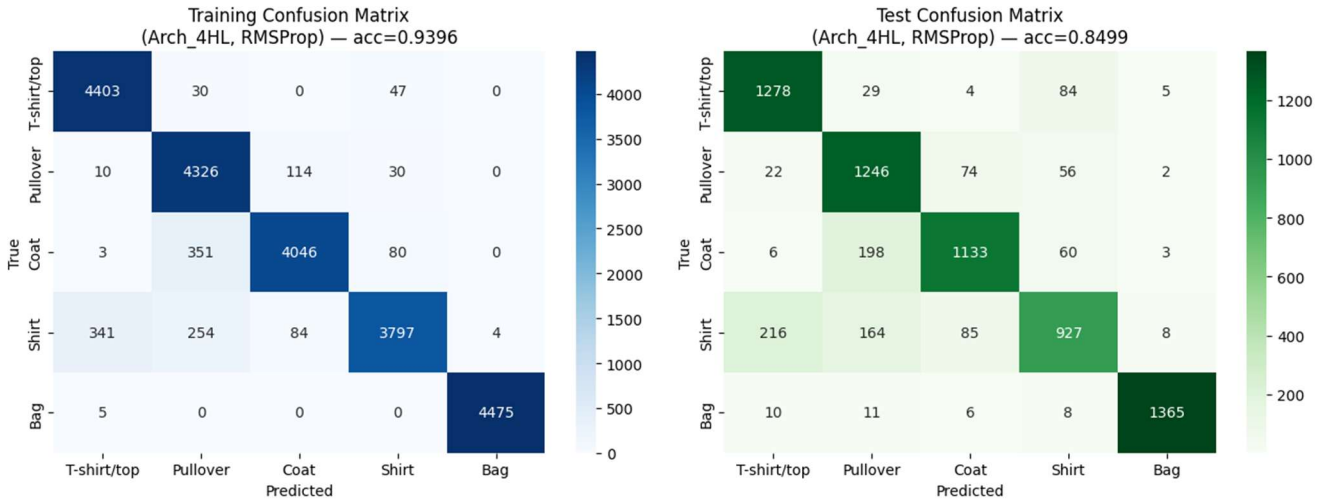


Figure 3. Training and test confusion matrices for Arch_4HL with RMSProp.

Training Confusion Matrix

True / Predicted	T-shirt/top	Pullover	Coat	Shirt	Bag
T-shirt/top	4403	30	0	47	0
Pullover	10	4326	114	30	0
Coat	3	351	4046	80	0
Shirt	341	254	84	3797	4
Bag	5	0	0	0	4475

Training accuracy: 93.96%.

Test Confusion Matrix

True / Predicted	T-shirt/top	Pullover	Coat	Shirt	Bag
T-shirt/top	1278	29	4	84	5
Pullover	22	1246	74	56	2
Coat	6	198	1133	60	3
Shirt	216	164	85	927	8
Bag	10	11	6	8	1365

Test accuracy: 84.99%.

4. Observations & Inferences

1. **Optimizer convergence and performance:** The six optimizers showed substantially different behavior. Batch Gradient Descent satisfied the loss-change stopping criterion after only two epochs for all architectures, but its validation accuracy remained between 7.96% and 19.98%, showing that numerical convergence does not necessarily imply effective learning. Ordinary SGD performed better but its validation accuracy decreased with depth (69.37% for Arch_3HL, 60.80% for Arch_4HL, and 51.64% for Arch_5HL), indicating that deeper networks were harder to optimize with a fixed learning rate of 0.001.

2. **Effect of Momentum-based and adaptive optimizers:** Momentum and NAG substantially improved upon ordinary SGD, achieving validation accuracies around 82–84%. NAG provided only a marginal improvement over Momentum. RMSProp and Adam performed best overall, demonstrating that adaptive parameter-wise learning rates were well suited to the high-dimensional Fashion-MNIST problem. RMSProp achieved the strongest validation performance, while Adam achieved the highest final training accuracy of 94.24% for Arch_5HL.

3. **Training–validation gap and generalization:** The models generally achieved much higher training than validation accuracy. For example, Arch_4HL with RMSProp reached 93.47% training accuracy compared with a peak validation accuracy of 85.93%, indicating a significant generalization gap. Thus, the highest training accuracy does not necessarily correspond to the best generalization performance.

4. **Effect of network depth:** Increasing depth did not produce a monotonic improvement in validation accuracy. The best observed result was achieved by **Arch_4HL + RMSProp**, with a peak validation accuracy of **85.93%**. Although Arch_5HL contains more than twice as many parameters as Arch_4HL, its best validation accuracies (85.64% with RMSProp and 85.70% with Adam) did not exceed Arch_4HL. This suggests that Arch_4HL provides a better balance between representational capacity and optimization complexity.

5. **Class-dependent classification errors:** The test confusion matrix shows that classification difficulty varies substantially across classes. **Bag** was the easiest class, with 1,365 of 1,400 test samples correctly classified, giving a recall of 0.97 and precision of 0.99. **Shirt** was the most difficult class, with a recall of 0.66; 216 Shirt images were classified as T-shirt/top and 164 as Pullover. Coat was also frequently confused with Pullover, reflecting the visual similarity among the garment categories.

6. **Methodological considerations:** The reported model-selection result uses the maximum validation accuracy observed during training (85.93%), but the notebook does not save a checkpoint at that best epoch; the final model was instead used for the 84.99% test evaluation. A more rigorous approach would save the best validation checkpoint before testing. Additionally, the implementation uses PyTorch nn.Linear, automatic differentiation, and built-in optimizers, while FAST_MODE=True uses mini-batches of 64 for SGD-family methods. These details should be acknowledged when describing the experimental methodology.